

Session 1: Introduction to Python, Jupyter Notebook, and Basic Syntax

Center for Data Analytics and Modeling — Professional Certificate for Data Science

LUMUMBA W. VICTOR

Chuka University
Chuka, Kenya

May 2026

Trainer: Lumumba W. Victor

Session Objectives

By the end of this session, you will be able to:

- 1 Understand the history and importance of Python in Data Science.
- 2 Set up the Python environment using Anaconda or pip.
- 3 Navigate the Jupyter Notebook interface efficiently.
- 4 Master basic Python syntax: Variables, Data Types, and Operators.
- 5 Implement control flow: Conditional Statements and Loops.
- 6 Perform basic linear algebra operations using NumPy (Vectors & Matrices).
- 7 Manipulate data using Pandas DataFrames.

A Brief History of Python

- **Creator:** Guido van Rossum.
- **Origin:** Late 1980s; First released in **1991**.
- **Design Philosophy:**
 - High-level, interpreted language.
 - Focus on **code readability** and simplicity.
 - Inspired by ABC, C, and Modula-3.
- **Evolution:** From a general-purpose language to the dominant force in:
 - Data Science & Statistical Analysis.
 - Machine Learning & AI.
 - Web Development & Automation.

Why Python for Data Science?

Simplicity & Readability

- English-like syntax reduces learning curve.
- Less boilerplate code compared to Java/C++.

Vast Ecosystem

- **NumPy:** Numerical computing.
- **Pandas:** Data manipulation.
- **Matplotlib/Seaborn:** Visualization.
- **Scikit-Learn:** Machine Learning.

Community Support

Python has one of the largest active developer communities, ensuring extensive documentation, tutorials, and third-party libraries.

What is Jupyter Notebook?

- An interactive, web-based environment for writing and executing Python code.
- Combines code, output, visualizations, and markdown text in a single document (.ipynb).
- Ideal for:
 - Exploratory Data Analysis (EDA).
 - Prototyping machine learning models.
 - Creating reproducible research reports.

Key Feature: Cell-based execution allows you to run code in chunks and see immediate results.

Installing Python & Jupyter

Method 1: Anaconda Distribution (Recommended)

- Comes with Python, Jupyter, and essential data science libraries pre-installed.
- Includes **Anaconda Navigator** for easy management.
- *Steps:* Download → Install → Launch Jupyter via Navigator.

Method 2: Standalone Python + pip

- Download Python from `python.org`.
- Install Jupyter manually via terminal/command prompt.
- Suitable for lightweight environments.

Installation via pip (Step-by-Step)

If using standalone Python, follow these steps:

❶ Download get-pip.py:

```
1 curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

❷ Install pip:

```
1 python get-pip.py
```

❸ Verify Installation:

```
1 pip --version
```

❹ Install Jupyter:

```
1 pip install jupyter notebook jupyterlab
```

Jupyter Notebook Interface Components

Cells: The basic unit of a notebook. Can be **Code** (executable) or **Markdown** (text/documentation).

Kernel: The computational engine that executes the code contained in the notebook.

Output: The result of executed code (text, plots, tables) appears directly below the cell.

Toolbar: Contains buttons for saving, adding cells, running cells, and changing cell types.

Tip: Always save your notebook frequently (Ctrl+S).

Basic Jupyter Commands (Shortcuts)

Efficiency comes from knowing your shortcuts. Use **Esc** to enter Command Mode.

Action	Shortcut
Run Cell	Shift + Enter
Insert Cell Below	Esc + B
Insert Cell Above	Esc + A
Delete Cell	Esc + D, D (Press D twice)
Change to Markdown	Esc + M
Change to Code	Esc + Y
Undo Cell Deletion	Esc + Z

Basic Syntax: Printing Output

The `print()` function is used to display output in Python.

Code:

Output:

```
1 print("Hello, world")  
2 print(5 + 10)
```

```
Hello, world  
15
```

- Strings must be enclosed in quotation marks.
- Mathematical expressions are evaluated before printing.

Data Types: Numbers

Python supports different numeric data types.

- 1 **Integer (int):** Whole numbers.

```
1 x = 20
2 print(type(x))
```

- 2 **Float (float):** Decimal numbers.

```
1 y = 3.14
2 print(type(y))
```

Note: Division using / always returns a float.

Data Types: Strings & Booleans

String (str): A sequence of characters.

```
1 name = "Mercy"  
2 greeting = "Hello"  
3  
4 print(name)  
5 print(greeting)
```

Boolean (bool): Represents truth values.

```
1 is_valid = True  
2 is_active = False  
3  
4 print(is_active)
```

- Boolean values are important in decision making.
- Python uses True and False.

Data Structure: Lists

List: An ordered and mutable collection of items.

```
1 fruits = ["apple", "banana", "orange"]
2
3 # Access elements
4 print(fruits[0])
5 print(fruits[2])
6
7 # Modify an element
8 fruits[1] = "blueberry"
```

- Lists use square brackets [].
- Items can be added, removed, or changed.

Data Structure: Tuples

Tuple: An ordered and immutable collection.

```
1 my_tuple = (1, 2, 3, "John", True)
2 coordinates = (10, 20)
3
4 # Access an element
5 print(coordinates[1])
6
7 # Tuples cannot be modified
8 # my_tuple[0] = 5
```

- Tuples use parentheses ().
- Useful for fixed data values.

Data Structure: Dictionaries

Dictionary (dict): Stores data as key-value pairs.

```
1 person = {  
2     "name": "John",  
3     "age": 25,  
4     "occupation": "Teacher"  
5 }  
6  
7 # Access a value  
8 print(person["name"])
```

- Dictionaries use curly braces {}.
- Keys must be unique.

Other Data Structures

Set: Collection of unique items.

```
1 my_set = {1, 2, 3, "John"}
```

None Type: Represents no value.

```
1 x = None
```

Arrays (NumPy): Used for efficient numerical computations.

Control Flow: Conditional Statements

Conditional statements allow decisions in Python.

```
1 age = 18
2
3 if age >= 18:
4     print("You are an adult")
5 else:
6     print("You are a minor")
```

Key Rules:

- Use if, elif, and else.
- Indentation is required in Python.
- Conditions return True or False.

Control Flow: Multiple Conditions

Use `elif` to test multiple conditions.

```
1 score = 85
2
3 if score >= 90:
4     print("Grade: A")
5 elif score >= 80:
6     print("Grade: B")
7 elif score >= 70:
8     print("Grade: C")
9 else:
10    print("Grade: F")
```

- Conditions are checked from top to bottom.
- Only the first true condition is executed.

Loops: The For Loop

A for loop iterates through a sequence.

```
1 # Loop through a range
2 for i in range(5):
3     print(i)
4
5 # Loop through a list
6 fruits = ["apple", "banana", "cherry"]
7
8 for fruit in fruits:
9     print(fruit)
```

`range(5)` generates numbers from 0 to 4.

Loops: The While Loop

A while loop repeats while a condition is true.

```
1 count = 0
2
3 while count < 3:
4     print(count)
5     count += 1
```

- Output will be 0, 1, and 2.
- Ensure the condition eventually becomes false.
- Otherwise, an infinite loop may occur.

Loop Control Statements

Loop control statements change loop behavior.

Break

```
1 for i in range(10):  
2     if i == 5:  
3         break  
4     print(i)
```

Continue

```
1 for i in range(5):  
2     if i == 2:  
3         continue  
4     print(i)
```

Pass

```
1 for i in range(3):  
2     if i == 1:  
3         pass  
4     print(i)
```

Advanced Looping Techniques

Enumerate: Returns both index and value.

```
1 fruits = ["apple", "banana"]  
2  
3 for index, fruit in enumerate(fruits):  
4     print(index, fruit)
```

Zip: Loops through multiple lists together.

```
1 names = ["Alice", "Bob"]  
2 scores = [85, 92]  
3  
4 for name, score in zip(names, scores):  
5     print(name, score)
```

Functions

Functions are reusable blocks of code.

```
1 def greet(name):  
2     return "Hello, " + name  
3  
4 message = greet("Alice")  
5 print(message)
```

- Functions are created using `def`.
- Parameters are inputs to a function.
- `return` sends back a result.

Mathematical Operations

Operation	Symbol	Example
Addition	+	$10 + 3 = 13$
Subtraction	-	$10 - 3 = 7$
Multiplication	*	$10 * 3 = 30$
Division	/	$10 / 3 = 3.33$
Floor Division	//	$10 // 3 = 3$
Modulus	%	$10 \% 3 = 1$
Exponentiation	**	$10 ** 3 = 1000$

```
1 import math
2
3 print(math.sqrt(25))
4 print(math.factorial(5))
```


Working with Vectors (NumPy)

Vector: A one-dimensional NumPy array.

```
1 import numpy as np
2
3 vector_a = np.array([1, 2, 3])
4 vector_b = np.array([4, 5, 6])
5
6 # Vector addition
7 sum_vec = vector_a + vector_b
8
9 # Dot product
10 dot_prod = np.dot(vector_a, vector_b)
```

- NumPy arrays are efficient for numerical work.
- They support fast mathematical operations.

Vector Operations Continued

Magnitude (Norm): Measures the length of a vector.

```
1 magnitude_a = np.linalg.norm(vector_a)
```

Scalar Multiplication:

```
1 scalar_mult = 3 * vector_a
```

Why Vectors?

Vectors are widely used in machine learning and data science to represent numerical features and perform mathematical operations efficiently.

Working with DataFrames (Pandas)

DataFrame: A two-dimensional table-like structure.

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Alice", "Bob", "Charlie"],
5     "Age": [25, 30, 35],
6     "County": ["Embu", "Nyeri", "Busia"]
7 }
8
9 df = pd.DataFrame(data)
10
11 print(df.head())
```

- DataFrames organize data into rows and columns.
- Each column can store different data types.

Basic DataFrame Operations

Accessing Data

```
1 # Access a column
2 df["Age"]
3
4 # Access first row
5 df.iloc[0]
```

Filtering and Sorting

```
1 # Filter rows
2 df[df["Age"] > 30]
3
4 # Sort values
5 df.sort_values(
6     by="Age",
7     ascending=False
8 )
```

Adding a New Column

```
1 df["Salary"] = [50000, 60000, 70000]
```

Handling Missing Data

Real-world datasets often contain missing values.

```
1 # Add a missing value
2 df.loc[2, "Salary"] = None
3
4 # Fill missing numerical values
5 df["Salary"].fillna(
6     df["Salary"].mean(),
7     inplace=True
8 )
9
10 # Fill missing categorical values
11 df["Occupation"].fillna(
12     df["Occupation"].mode()[0],
13     inplace=True
14 )
```

- Missing values are represented as NaN.
- Data cleaning is important before analysis.

Summary & Next Steps

Recap

- Python is simple, readable, and powerful for data science.
- Jupyter Notebooks allow interactive coding and documentation.
- Mastered basic data types (Lists, Dicts, Tuples) and control flow (If/Loops).
- Introduced NumPy for vectors and Pandas for DataFrames.

Next Session: Deep dive into Pandas for Data Cleaning and Preparation.

Questions?